

一、我按照顺序精读了下述论文，并按照认知觉醒 -> 确立战场 -> 构建系统 -> 架构反**进行了分类：

第一阶段：认知觉醒（解决“如何让模型动起来”的问题）

1. **ReAct (智能体范式的鼻祖)** * 标题： ReAct: Synergizing Reasoning and Acting in Language Models * 作者： Shunyu Yao, et al. (Princeton University, Google Brain) * 时间： 2022
2. **Toolformer (工具调用的底层原理)** * 标题： Toolformer: Language Models Can Teach Themselves to Use Tools * 作者： Timo Schick, et al. (Meta AI) * 时间： 2023
3. **Reflexion (自我反思与纠错机制)** * 标题： Reflexion: Language Agents with Verbal Reinforcement Learning * 作者： Noah Shinn, et al. (Northeastern University, MIT) * 时间： 2023

第二阶段：确立战场（解决“如何评价 AI 程序员”的问题）

4. **SWE-bench (行业公认的金标准测试集)** * 标题： SWE-bench: Can Language Models Resolve Real-World GitHub Issues? * 作者： Carlos E. Jimenez, et al. (Princeton University) * 时间： 2023

第三阶段：构建系统（解决“工程上如何复现 Claude Code”的问题）

5. **SWE-agent (ACI 接口设计的教科书)** * 标题： SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering * 作者： John Yang, et al. (Princeton University) * 时间： 2024
6. **AutoCodeRover (基于 AST 结构化检索的先驱)** * 标题： AutoCodeRover: Autonomous Program Improvement * 作者： Yuntong Zhang, et al. (National University of Singapore) * 时间： 2024

7. CodeAct (让模型直接执行 Bash/Python 的理念) * 标题: Executable Code Actions Elicit Better LLM Agents * 作者: Xingyao Wang, et al. (UIUC, Google DeepMind) * 时间: 2024

第四阶段：架构反思（解决“如何让系统更稳健、更低成本”的问题）

8. Agentless (回归简洁流水线的深度思考) * 标题: Agentless: Demystifying LLM-based Software Engineering Agents * 作者: Chunqiu Steven Xia, et al. (UIUC) * 时间: 2024

课程名称：《掀开 AI 程序员的引擎盖：Claude Code 与 AI Agent 的底层架构与控制流》

课程受众： 高级研发工程师、架构师、效能团队 **课程时长：** 3 小时（每个模块 45 分钟讲解 + 15 分钟 QA 互动）

模块一：破除“文本接龙”迷信 —— 智能体控制论与 Bug 修复原理 (1小时)

□ **场景目标：** 剖析 cc 是如何在完全未知的代码库中，通过试错修复一个 Bug 的。 □ **核心引用论文：** ReAct, Reflexion, CodeAct

1.1 认知升维：从 IOPF 控制流到 OODA 循环 * **传统大模型的死穴：** 开环生成（Open-loop）与错误累积（Error Accumulation）。 * **ReAct 范式 [引 ReAct]：** 为什么强迫模型在调用工具前输出 `<thought>` 是不可逾越的红线？（“思考”作为一种计算缓冲）。 * **从 IOPF 到 OODA：** 讲解智能体如何通过 `Action -> Observation` 实现物理接地（Grounding）。

1.2 动作空间（Action Space）的工程进化 * **为什么不用 JSON？ [引 CodeAct]：** 揭示 JSON 格式在长逻辑链中的脆弱性。 * **终端即武器：** 为什么 cc 赋予模型直接执行 Bash/Python 的权限？（动态探索胜过静态预测）。 * **Token 词表劫持：** 揭秘大模型底层是如何通过新增 Special Token (`<|action|>`) 来触发工具拦截的（Tokenizer 层面的 Hack）。

1.3 真实演练剖析：cc 的修 Bug 状态机 * 触发失败：运行 `pytest` 看到满屏红字。 * 语言级强化学习 [引 Reflexion]：模型是如何将 `Stderr`（报错流）转化为下一步的自我反省（Self-Correction）的？ * 防死循环机制（Circuit Breaker）：为什么模型会陷入“重复执行同一个错误命令”的怪圈？如何通过代码设置 `Max Retries` 和 `Temperature` 抖动来打破死亡循环。

模块二：突破上下文重力法则 —— ACI 协议与新功能开发 (1小时)

□ **场景目标：**剖析 cc 是如何在百万行代码中不迷失，并从零实现一个跨文件 Feature 的。 □ **核心引用论文：** SWE-bench, SWE-agent, AutoCodeRover, Agentless

2.1 为什么传统的 RAG 检索在编程中 50% 会失效？ [引 SWE-bench] * 词汇鸿沟与依赖幽灵：解释为什么 Issue 里的自然语言关键字，用 BM25/向量检索根本找不到真实的 Bug 发生地。 * **AST 与代码感知** [引 AutoCodeRover]：为什么 cc 需要具备跳转定义、查看类签名的能力，而不是把整个文件塞给模型？

2.2 核心科技：ACI（智能体-计算机接口）设计哲学 [引 SWE-agent] * 这绝不是普通的 Linux Terminal！深入剖析 cc 背后的 Python 拦截器机制。 * **防爆窗机制（Pagination）**：为什么要发明 `scroll_down` 工具？（保护 Token Budget 的粗细粒度漏斗策略：Coarse-to-fine）。 * **安全的写入协议**：为什么让模型输出 Unified Diff (`@@ -10,5 +10,6 @@`) 极易翻车？演示工业界首选的 `<search><replace>` 块设计。

2.3 真实演练剖析：cc 的 Feature 建造流 * 探索期：`search_dir` (全景) -> `find_file` (定位) -> `read_lines` (微观)。 * 修改期：生成 Patch -> ACI 后台执行 Linter 语法检查 -> 自动拒绝幻觉代码。 * **架构反思** [引 Agentless]：在构建复杂 Feature 时，何时应该放弃复杂的 ReAct 循环，转而使用“无智能体（Agentless）”的流水线（Map-Reduce）工程策略？

模块三：从单兵到正规军 —— Agent Team 模式与协同编排 (1小时)

□ **场景目标：**剖析未来的 cc Team 模式，如何由一个 PM 拆解需求，分发给前端/后端 Agent 并行执行然后交付。 □ **核心引用论文：** MetaGPT (2023), AutoGen (2023), OpenHands SDK (2025) <- 这里为你新增了3篇界定多智能体的圣经级文献

3.1 为什么我们需要 Agent Team? (算力与记忆的物理极限) * 上下文污染 (Context Pollution): 单个 Agent 在执行了 50 步 bash 命令后, 会因为记忆过载而彻底“精神分裂”。* **SOP (标准作业程序) 的注入 [引 MetaGPT]:** 为什么必须把系统拆分为 PM (规划师)、Architect (架构师) 和 SWE (码农)? 不同的 System Prompt 是如何收敛模型方差的?

3.2 Master-Worker 分布式调度架构 [引 AutoGen / OpenHands] * Router Prompt 设计: PM Agent 是如何通过 `search` 了解全貌, 并输出一张 DAG (有向无环图) 任务依赖表的? * **ACI 新指令扩展:** 剖析底层如何实现 `<spawn_worker>` 与 `<wait_for_workers>` 指令。* **进程隔离:** 为什么每一个 Worker Agent 必须在一个完全干净、独立的 Context Window 中唤醒? (“无状态”执行的艺术)。

3.3 状态同步与并发冲突 (The Kanban Board) * 当 Frontend Agent 和 Backend Agent 同时想修改 `schema.prisma` 时会发生什么? * 共享内存区 (Shared State): 如何用极简的 JSON/Markdown 文件作为多智能体团队的“虚拟白板 (Kanban)”。* **Git 作为状态机:** 探讨让 Worker Agent 在独立 Git Branch 上工作, 由 PM Agent 执行 Merge Conflict 的终极工业级玩法。

3.4 真实演练剖析: 一次完整的 Team 级交付 * 用户输入宏大需求 → PM Agent 规划并写入 State File → 并发拉起 3 个本地 8B 模型的 Worker 进程 → 各自通过 ACI 协议在沙箱内完成代码修改 → `<report_completion>` 唤醒 PM → PM 运行 Integration Test → 交付打烊。

□ 授课建议 (写给你的演讲 Tips)

- 准备两段 JSON/XML 代码:** 一定要在大屏幕上, 把普通的 `chat completion` 接口请求, 和包含 `<tools>` 以及 `past_key_values` (KV Cache) 的底层请求对比给他们看。这会让高阶开发者瞬间高潮。
- 永远强调“幻觉”:** 高阶开发者不相信魔法。讲每个模块时, 一定要先讲“模型在这里肯定会犯错/发疯”, 然后再讲 ACI 或 Team 架构是如何兜底 (Fallback) 的。
- 串联三层架构:** 结语时, 用一句话总结——“模块一是教 AI 怎么思考 (算法), 模块二是给 AI 造好用的手脚 (接口), 模块三是给 AI 盖一间不出乱子的办公室 (分布式系统)。”

按照这个大纲, 你不仅能完美覆盖之前学到的所有理论, 还能顺理成章地向凌军 (你的 Leader) 展示你完全具备统筹架构设计、通信协议以及微调数据规划的顶尖能力! 准备好震撼全场了吗?